

30538 Problem Set 4: Web Scraping

Julio Pacher

2026-02-07

Due 02/07 at 5:00PM Central.

“This submission is my work alone and complies with the 30538 integrity policy.” Add your initials to indicate your agreement: **JP**

Github Classroom Assignment Setup and Submission Instructions

1. Accepting and Setting up the PS4 Assignment Repository

- Each student must individually accept the repository for the problem set from Github Classroom (“ps4”) – <https://classroom.github.com/a/hWhcHqH>
 - You will be prompted to select your cnetid from the list in order to link your Github account to your cnetid.
 - If you can’t find your cnetid in the link above, click “continue to next step” and accept the assignment, then add your name, cnetid, and Github account to this Google Sheet and we will manually link it: <https://rb.gy/9u7fb6>
- If you authenticated and linked your Github account to your device, you should be able to clone your PS4 assignment repository locally.
- Contents of PS4 assignment repository:
 - `ps4_template.qmd`: this is the Quarto file with the template for the problem set. You will write your answers to the problem set here.

2. Submission Process:

- Knit your completed solution `ps4.qmd` as a pdf `ps4.pdf`.
 - Your submission does not need runnable code. Instead, you will tell us either what code you ran or what output you got.
- To submit, push `ps4.qmd` and `ps4.pdf` to your PS4 assignment repository. Confirm on Github.com that your work was successfully pushed.

Grading

- You will be graded on what was last pushed to your PS4 assignment repository before the assignment deadline
- Problem sets will be graded for completion as: {missing (0%); - (incomplete, 50%); + (excellent, 100%)}
 - The percent values assigned to each problem denote how long we estimate the problem will take as a share of total time spent on the problem set, not the points they are associated with.
- In order for your submission to be considered complete, you need to push both your `ps4.qmd` and `ps4.pdf` to your repository. Submissions that do not include both files will automatically receive 50% credit.

```

import pandas as pd
import altair as alt
import time
import requests
from bs4 import BeautifulSoup
import re
from urllib.parse import urljoin
from datetime import datetime

import warnings
warnings.filterwarnings('ignore')
alt.renderers.enable("png")

```

```

RendererRegistry.enable('png')

```

Step 1: Develop initial scraper and crawler

```

BASE_URL = "https://oig.hhs.gov"
START_URL = "https://oig.hhs.gov/fraud/enforcement/"
HEADERS = {"User-Agent": "DAP30538CourseBot/1.0 (jpacher@uchicago.edu)"}
DATE_PAT = re.compile(r"^[A-Z][a-z]+ \d{1,2}, \d{4}$")

```

```

resp = requests.get(START_URL, headers=HEADERS, timeout=30)

resp.raise_for_status()

soup = BeautifulSoup(resp.text, "html.parser") #could not install lxml

rows = []

```

```

for a in soup.find_all("a", href=True):
    href = a["href"]

    if not (href.startswith("/fraud/enforcement/") and href !=
        ↪ "/fraud/enforcement/"):
        continue

    title = a.get_text(strip=True)
    link = urljoin(BASE_URL, href)

```

```

container = a.find_parent(["li", "article", "div"])
if container is None:
    continue

text_lines = [t.strip() for t in container.stripped_strings if t.strip()]

date = None
date_idx = None
for i, t in enumerate(text_lines):
    if DATE_PAT.match(t):
        date = t
        date_idx = i
        break

categories = []
if date_idx is not None:

    for t in text_lines[date_idx + 1:]:
        if t == title:
            continue

        categories.append(t)

category = "; ".join(categories) if categories else None

if date is None:
    continue

rows.append({"title": title, "date": date, "category": category, "link":
↪ link})

df = pd.DataFrame(rows).drop_duplicates(subset=["title", "date",
↪ "link"]).reset_index(drop=True)

print(df.head())

```

	title	date \
0	Houston Transplant Doctor Indicted For Making ...	February 5, 2026
1	MultiCare Health System to Pay Millions to Set...	February 4, 2026
2	Brooklyn Banker Pleads Guilty to Laundering Pr...	February 3, 2026
3	Delafield Man Sentenced to 18 Months' Imprison...	February 3, 2026
4	Former NFL Player Convicted for \$197M Medicare...	February 3, 2026

```

                category \
0 Criminal and Civil Actions
1 Criminal and Civil Actions
2 COVID-19
3 Criminal and Civil Actions
4 Criminal and Civil Actions

                link
0 https://oig.hhs.gov/fraud/enforcement/houston-...
1 https://oig.hhs.gov/fraud/enforcement/multicar...
2 https://oig.hhs.gov/fraud/enforcement/brooklyn...
3 https://oig.hhs.gov/fraud/enforcement/delafiel...
4 https://oig.hhs.gov/fraud/enforcement/former-n...

```

```

display(df.columns)
display(df.shape)

```

```

Index(['title', 'date', 'category', 'link'], dtype='object')

(20, 4)

```

Step 2: Making the scraper dynamic

1. Turning the scraper into a function

- a. Pseudo-Code
- Print a message if the input year is less than 2013 and stop execution.
- Create a cutoff date corresponding to the first day of the requested month and year.
- Initialize page = 1 and an empty list to store all scraped enforcement actions.
- Use a while loop to crawl pages:
 - Fetch the HTML for the current page and locate enforcement action cards.
 - If no cards are found, stop crawling.
 - For each card, extract the title, date, category, and link, convert the date to a comparable format, and store the result.
 - If the oldest date on the page is earlier than the cutoff date, stop crawling.
 - Otherwise, wait briefly and move to the next page.

- Convert the collected records into a tidy dataframe, save it as enforcement_actions_year_month.csv, and return the dataframe.

(I used AI to help me write my pseudo code, I first write it up and after I asked him to correct it.)

- b. Create Dynamic Scraper

```
def crawl_enforcement_actions(start_month: int, start_year: int,
    ↪ sleep_seconds: int = 1) -> pd.DataFrame:
    """Crawl pages until reaching start_month/start_year, then return rows
    ↪ since cutoff."""
    if start_year < 2013:
        print("Please use year >= 2013. Enforcement actions are listed from
        ↪ 2013 onward.")
        return pd.DataFrame(columns=["title", "date", "category", "link",
        ↪ "date_parsed"])

    cutoff = datetime(start_year, start_month, 1)

    page = 1
    all_pages = []

    while True:
        df_page = parse_one_page(page)

        if df_page.empty:
            break

        df_page["date_parsed"] = pd.to_datetime(df_page["date"], format="%B
    ↪ %d, %Y", errors="coerce")
        all_pages.append(df_page)

        oldest_on_page = df_page["date_parsed"].min()
        if pd.isna(oldest_on_page) and oldest_on_page < cutoff:
            break

        time.sleep(sleep_seconds)
        page += 1

    if not all_pages:
        return pd.DataFrame(columns=["title", "date", "category", "link",
        ↪ "date_parsed"])
```

```

df_all = pd.concat(all_pages, ignore_index=True)
df_all = df_all[df_all["date_parsed"] >= cutoff].copy()
df_all = df_all.sort_values("date_parsed",
↪ ascending=False).reset_index(drop=True)

return df_all

RUN_SCRAPER = False

start_month = 1
start_year = 2024
csv_name = f"enforcement_actions_{start_year}_{start_month:02d}.csv"

if RUN_SCRAPER:
    df_2024 = crawl_enforcement_actions(start_month, start_year,
↪ sleep_seconds=1)
    df_2024.to_csv(csv_name, index=False)
else:
    df_2024 = pd.read_csv(csv_name)
    df_2024["date_parsed"] = pd.to_datetime(df_2024["date"], format="%B %d",
↪ %Y", errors="coerce")

#df_2024.head()

```

```

print(len(df_2024))
earliest = df_2024.sort_values("date_parsed", ascending=True).iloc[0]
earliest_details = earliest[["date", "title", "category", "link"]]
print(earliest_details)

```

```

1787
date                January 3, 2024
title              Former Nurse Aide Indicted In Death Of Clarksv...
category            State Enforcement Agencies
link               https://oig.hhs.gov/fraud/enforcement/former-n...
Name: 1786, dtype: object

```

- c. Test Your Code

```

RUN_SCRAPER = False

start_month = 1

```

```

start_year = 2022
csv_name = f"enforcement_actions_{start_year}_{start_month:02d}.csv"

if RUN_SCRAPER:
    df_main = crawl_enforcement_actions(start_month, start_year,
    ↪ sleep_seconds=1)
    df_main.to_csv(csv_name, index=False)
else:
    df_main = pd.read_csv(csv_name)
    df_main["date_parsed"] = pd.to_datetime(df_main["date"], format="%B %d,
    ↪ %Y", errors="coerce")

```

```
df_main["date_parsed"].min(), df_main["date_parsed"].max()
```

```
(Timestamp('2022-01-04 00:00:00'), Timestamp('2026-02-05 00:00:00'))
```

```

print(len(df_main))
earliest_main = df_main.sort_values("date_parsed", ascending=True).iloc[0]
earliest_details_main = earliest_main[["date", "title", "category", "link"]]
print(earliest_details_main)

```

```

3377
date                                January 4, 2022
title      Integrated Pain Management Medical Group Agree...
category                                Fraud Self-Disclosures
link      https://oig.hhs.gov/fraud/enforcement/integrat...
Name: 3376, dtype: object

```

Step 3: Plot data based on scraped data

```

# Load the CSV created in Step 2
df = pd.read_csv(
    "enforcement_actions_2022_01.csv",
    parse_dates=["date"]
)

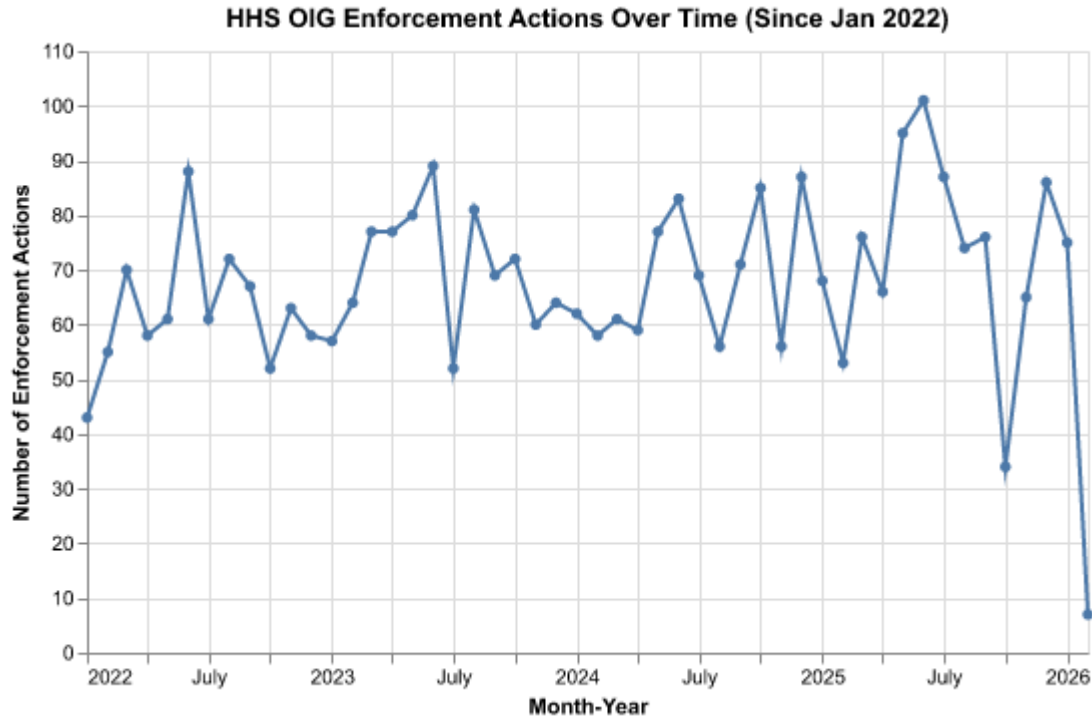
# Create year-month variable for aggregation
df["year_month"] = df["date"].dt.to_period("M").dt.to_timestamp()

#df.head()

```

1. Plot the number of enforcement actions over time

```
monthly_counts = (  
    df.groupby("year_month")  
        .size()  
        .reset_index(name="n_actions")  
)  
  
chart_total = (  
    alt.Chart(monthly_counts)  
        .mark_line(point=True)  
        .encode(  
            x=alt.X("year_month:T", title="Month-Year"),  
            y=alt.Y("n_actions:Q", title="Number of Enforcement Actions"),  
            tooltip=["year_month:T", "n_actions:Q"]  
        )  
        .properties(  
            title="HHS OIG Enforcement Actions Over Time (Since Jan 2022)",  
            width=500,  
            height=300  
        )  
)  
  
chart_total
```



2. Plot the number of enforcement actions categorized:

- based on “Criminal and Civil Actions” vs. “State Enforcement Agencies”

```
df["main_category"] = df["category"].where(
    df["category"].isin(["Criminal and Civil Actions", "State Enforcement
↪ Agencies"])
)

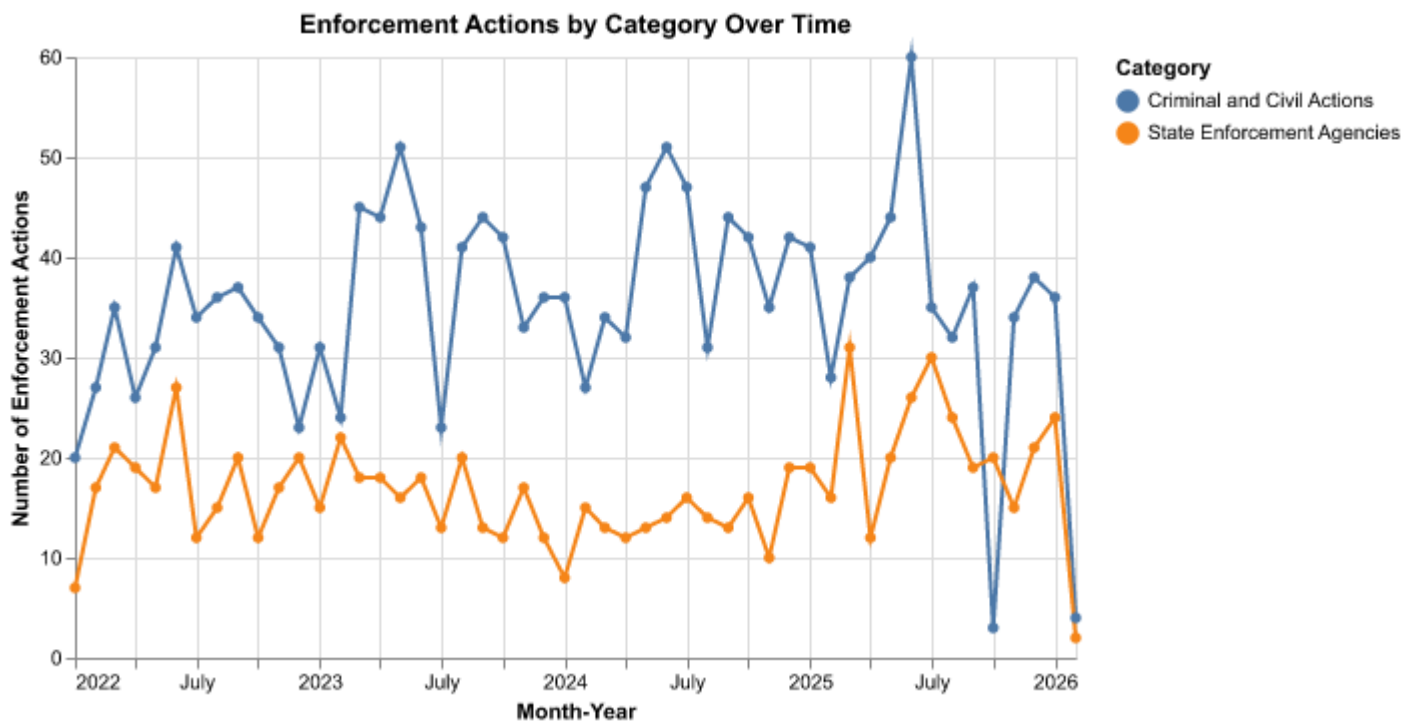
monthly_cat = (
    df.groupby(["year_month", "main_category"])
      .size()
      .reset_index(name="n_actions")
)
```

```
chart_category = (
    alt.Chart(monthly_cat)
      .mark_line(point=True)
      .encode(
        x=alt.X("year_month:T", title="Month-Year"),
```

```

y=alt.Y("n_actions:Q", title="Number of Enforcement Actions"),
color=alt.Color("main_category:N", title="Category"),
tooltip=["year_month:T", "main_category:N", "n_actions:Q"]
)
.properties(
    title="Enforcement Actions by Category Over Time",
    width=500,
    height=300
)
)
chart_category

```



- based on five topics

```

def classify_topic(title):
    t = title.lower()

    if any(k in t for k in ["health", "medicare", "medicaid", "hospital",
        ↪ "physician", "clinic"]):

```

```

        return "Health Care Fraud"
    elif any(k in t for k in ["bank", "financial", "loan", "credit", "wire",
        ↪ "securities"]):
        return "Financial Fraud"
    elif any(k in t for k in ["drug", "opioid", "pharma", "controlled
        ↪ substance"]):
        return "Drug Enforcement"
    elif any(k in t for k in ["bribe", "bribery", "corrupt", "kickback"]):
        return "Bribery/Corruption"
    else:
        return "Other"

```

```

df_cc = df[df["category"] == "Criminal and Civil Actions"].copy()
df_cc["topic"] = df_cc["title"].apply(classify_topic)

monthly_topics = (
    df_cc.groupby(["year_month", "topic"])
        .size()
        .reset_index(name="n_actions")
)

```

```

chart_topics = (
    alt.Chart(monthly_topics)
        .mark_line(point=True)
        .encode(
            x=alt.X("year_month:T", title="Month-Year"),
            y=alt.Y("n_actions:Q", title="Number of Enforcement Actions"),
            color=alt.Color("topic:N", title="Topic"),
            tooltip=["year_month:T", "topic:N", "n_actions:Q"]
        )
        .properties(
            title="Criminal and Civil Enforcement Actions by Topic",
            width=500,
            height=300
        )
)

chart_topics

```

